

RESEARCH

Open Access



Riskquant-grl: a self-optimal control method for power network communication based on graph deep reinforcement learning

Yingjie Jiang¹, Yongjing Wei¹, Chao Sun^{1*}, Anqi Tian¹, Yan Zhang¹ and Youxiang Zhu¹

*Correspondence:

Chao Sun

15866676429@163.com

¹State Grid Shandong Electric Power Company Information and Communication Company, Jinger Road, Jinan, Shandong 250002, China

Abstract

Under the framework of smart grid development, electric power communication networks face challenges in real-time operations, transmission reliability, and intelligent management. This paper proposes a self-convergent optimization control strategy, RiskQuant-GRL (RQGRL), which integrates risk quantization and graph deep reinforcement learning. By combining Graph Neural Networks (GNN) with Deep Q-Learning (DQN), RQGRL aims to enhance network intelligence in resource scheduling and path planning. Specifically, it embeds a Graph Convolutional Network (GCN) into a Deep Reinforcement Learning (DRL) framework to monitor the network state, make intelligent decisions, and provide optimal routing. By quantifying risk as edge weights and using weighted GCN, the model dynamically optimizes path selection and resource allocation. Simulation results show that RQGRL improves bandwidth allocation efficiency by 6.6% over state-of-the-art DRL solutions and excels in link failure recovery, maintaining efficient network operation even under high concurrency.

Keywords Deep reinforcement learning, Graph neural network, Deep Q-learning algorithms, Path recommendation

1 Introduction

Under the background of rapid development of information technology, electric power communication network, acting as the nervous system of energy internet, carries the key functions of power grid monitoring, data transmission and order giving, and its stability and efficiency directly affect the operation safety and service quality of the whole power grid. Fiber optic communication technology, with its advantages of large capacity, high speed and low loss, occupies a core position in the power communication system. However, with the in-depth development of the smart grid, higher requirements have been put forward for the real-time, reliability, and intelligence level of the communication network, especially in the face of complex scenarios such as large-scale distributed power generation, multi-energy complementation, and power market trading, the traditional network management and optimization strategies gradually show their limitations



© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

[1, 2]. In addition, the solutions obtained by heuristic algorithms are far from optimal. In the face of this series of challenges, how to efficiently and dynamically perform resource scheduling to ensure the stable operation of the network under various working conditions has become a key problem to be solved.

Especially in terms of uneven resource allocation, frequent network congestion, and fast response and recovery capability in the face of sudden failures, the existing technologies are difficult to meet the growing demand for intelligent management. Meanwhile, DRL has made significant progress in sequential decision making and automatic control problems [3, 4]. As a result, the networking community has investigated DRL as a key technique for network optimization (e.g., routing) aiming at the realization of self-driving networks [5–8]. In this context, it is particularly urgent and important to introduce the cutting-edge results of DRL into the field of power communication network management and optimization [9]. The current research status of power communication networks shows that although some work has been focused on using advanced path planning algorithms to enhance network efficiency and stability [10–12], most of these studies focus on optimizing data transmission paths and fail to fully consider the risk factors in network operation, including but not limited to potential bottlenecks triggered by uneven resource allocation, degradation of quality of service due to network congestion, and vulnerability and recovery in the face of unexpected events. This neglect of risk assessment limits the effectiveness of management strategies and the overall resilience of the network.

Given the above research limitations, this study aims to break through the framework of traditional methods and propose a novel self-optimizing control strategy for power communication networks. The strategy innovatively incorporates the risk quantification method and utilizes the powerful function of DRL to realize real-time monitoring and intelligent decision-making of the network state. The main contributions of this paper include:

- We propose a self-optimizing control method for power communication based on risk quantization and deep reinforcement learning, which targetedly solves the problem of risk neglect in path planning and resource scheduling in power communication networks. The method utilizes graph convolutional network (GCN) to deeply extract and comprehensively analyze the features of the environment around the nodes, which not only focuses on the traffic scheduling among the nodes, but also introduces the risk assessment factor, and the core of the method lies in the deep feature mining of the nodes and their connectivity characteristics in the network through the GCN model to obtain a more accurate representation of the state. On this basis, a multidimensional evaluation system including risk quantification is designed to guide the deep reinforcement learning algorithm (DRL) to make intelligent decisions.
- The postgraph neural grid is embedded into the reinforcement learning framework to make full use of the adaptation of reinforcement learning to uncertainties for fast solution while using fiber optic to transmit the structural information of the grid structure. To overcome the local optimal problem that may be caused by the greedy strategy in the traditional reinforcement learning, this method introduces the Experience Replay Buffer (ERB). By replaying and evaluating the historical path selection over some time (i.e., within the temporal period of a time node),

this mechanism helps to learn the long-term optimization strategy. It avoids the impact of short-term greedy behavior on the overall path selection optimization. The experience replay pool not only enhances the robustness of the learning process but also improves the model's ability to adapt to the complex network environment, ensuring that the path selection meets both immediate needs and long-term interests.

- Compared with the traditional methods, the method proposed in this paper provides a significant improvement in routing and traffic distribution, which can effectively identify and avoid the affected links and maintain better performance than the theoretical baseline.

2 Related work

2.1 Graph neural network-based risk situational awareness

Risk situational awareness refers to the ability to continuously monitor and analyze the internal and external environment of the network, identify potential threats, assess security risks, and predict possible future security situations in the management of electric power communication networks [13]. In the smart grid context, the power communication network carries critical monitoring and control information, and its stability and efficiency directly affect the operational security of the power grid [14]. Risk situational awareness enables network managers to identify the most vulnerable links and allocate resources effectively, thereby enhancing the resilience and reliability of the overall network. Extracting and learning from potential risks in the power communication network remains a critical challenge. DU et al. [15] propose a transient stability assessment method leveraging bi-directional gated recurrent units (BGRUs), which enhance the learning capabilities for challenging and unstable samples. BGRUs are a variant of long short-term memory (LSTM) neural networks. Graph neural networks [16], as a model that can effectively deal with data sets containing complex developed many variants since [17, 18], and their characteristics are very suitable for feature processing in power communication networks [19, 20]. Zhang et al. [21] utilize graph neural networks (GNNs) to address uncertainties by designing pre-GNNs and post-GNNs embedded within a reinforcement learning framework. This approach enhances the model's adaptability to changes in target topology and improves its generalization and computational efficiency. Guo et al. [22] uses reinforcement learning-based graph neural networks to adapt to the complex grid state changes in a new type of power system to adjust and optimize the grid strategy in real-time. Adjust and optimize the grid strategy in real-time. However, the computational accuracy and efficiency of the above literature still need to be further improved when dealing with the complex topology, dynamically changing characteristics, and high-dimensional risk indicators of power grids. Meanwhile, the system considers fewer risk factors, which limits the comprehensiveness and practicality of the model to some extent. To address the above problems, this paper proposes a new method that not only considers the basic risk factors but also introduces emergency treatment plans for these risk factors, etc. By quantifying the various risk factors and integrating them into a comprehensive risk score, and presenting this feature into the DRL algorithm, it enhances the ability to perceive and cope with the various risk factors in the complex power grid environment.

2.2 Path selection and bandwidth allocation in synchronous digital systems

Synchronous Digital Hierarchy (SDH) is an international standard fiber-optic communication system for efficient transmission, multiplexing, routing, and management of multiple data types. SDH, as one of the core transmission technologies of power communication networks, provides stable and reliable data transmission service for the power grid with its highly standardized, flexible, and reliable features. With the rapid growth of data traffic generated by the continuous development of smart grids, it is an ongoing challenge to efficiently utilize network resources, especially to achieve optimal bandwidth allocation under dynamically changing traffic demands. Reinforcement learning (RL) [23] has been recognized as an effective approach for achieving dynamic allocation of network resources. For instance, José et al. [24] explored the application of deep reinforcement learning (DRL) for online routing in optical transport networks. They proposed a fine-grained representation of network states and designed an intelligent policy to optimize long-term resource conservation. Similarly, Paul et al. [11] integrated graph neural networks (GNNs) into a DRL agent, designing a problem-specific action space that enhances the model's ability to generalize across arbitrary network topologies. However, the above methods are still insufficiently considered when dealing with risk factors. In optical transport networks, risk factors such as link failures, node failures, and bursty traffic have a significant impact on network performance and stability. Traditional methods tend to focus on optimizing resource utilization and bandwidth allocation without considering these potential risk factors, which may lead to unexpected network performance degradation or disruption in practical applications. In this paper, we propose a novel approach to address uncertainties in network decision-making by quantifying risks and incorporating them as features. Specifically, we define a set of risk assessment metrics, such as link failure rate, packet loss rate, and contingency plan availability. These metrics are weighted and aggregated to generate a comprehensive risk score, which reflects the potential risks of the current network state and serves as critical input for DRL agents during decision-making.

3 Self-optimizing control model for power communication networks

The self-optimizing control model of the power communication network mainly includes a feature extraction module and a GRL agent module. Firstly, the actual optical transmission network is abstracted into graph data, which contains the network topology of SDH and its fiber optic cable feature data. The abstracted data is inputted to graph convolutional network (GCN) for node feature extraction, and the node feature vector is output. Then, the feature vectors of the nodes at the two ends of the branch and the branch features are spliced and input to the branch feature extraction network consisting of the fully connected layer to output the branch feature vector. Finally, the branch feature vector is input to the rectified linear unit (ReLU) to output the risk analysis result of the branch. This risk analysis result is quantized as the weights of the branches. Two layers of weighted GCNs are embedded in each of the eval net and target net of the downstream deep Q-network framework to extract further the grid information and the quantized information of the risk analysis and perform the weighted analysis. These two layers of GCNs will be updated together with the DQN, and the final action value at time t is output from the eval net. The intelligent body executes the action, the environment state switches from the current state to the next state, and the current action reward

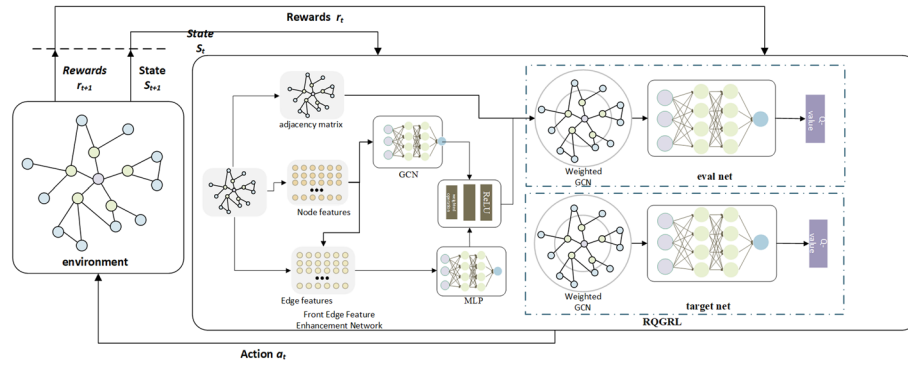


Fig. 1 Self-convergent optimal control model for electric power communication networks

Table 1 Node and Branch Feature Vectors

Type	Feature	Meaning
Node	t_u	Node type
Node	s_u	Device status
Node	c_u	Capacity
Node	p_u	Packet loss rate
Node	e_u	Emergency level
Branch	b	Bandwidth
Branch	ρ	Signal quality
Branch	a	Backup path
Branch	B_{uv}	Link betweenness

value is fed back for the intelligent body to learn. Figure 1 shows the model architecture diagram of this paper.

3.1 Problem definition and preparatory knowledge

3.1.1 Problem definition

In this paper, the fiber optic communication network is abstracted as an SDH topology graph, where: V denotes the set of all nodes and ϵ denotes the set of all branches. The feature vectors referenced by the model for the problem posed in this paper include: the feature vectors of the nodes are X_u and contain the node type t_u , equipment status s_u , capacity c_u , packet loss rate p_u , and the level of contingency plans supported by the nodes e_u . The feature vectors of the branches $u \rightarrow v$ include bandwidth b and signal quality q and a binary marker for whether an alternate path is configured as a contingency a . The link median formula denoting the branch can be expressed as $B_{uv} = \sum_{p \neq u \neq v \neq q} \frac{\sigma(uv)}{\sigma_{pq}}$. Where B_{uv} denotes the number of link meshes, $\sigma(uv)$ is the number of links passing through the link from all the shortest paths from node p to node q . and σ_{pq} is the total number of all the shortest paths from p to q . Table 1 shows the types of features required in this paper.

3.1.2 Background knowledge

In this paper, we employ the DQN algorithm to train out model. Below, we introduce the fundamental principles of DQN. The theoretical foundation of the DQN algorithm is rooted in the Bellman equation for Q-Learning [25]. However, its key innovation lies in utilizing a deep neural network as an approximator for the Q-value function. Specifically,

for any given state s and action a , the goal of DQN is to learn a parameterized function $Q(s, a; \theta)$, where θ denotes the parameters of the neural network

$$Q^*(s, a) = \mathbb{E} \left[R(s, a, s') + \gamma \max_{a'} Q(s', a') \mid s, a \right] \quad (1)$$

Here, $R(s, a; \theta)$ represents the expected immediate reward received after transitioning from state s to state s' by taking action a , and γ is the discount factor that determines the importance of future rewards

DQN updates the network parameters by minimizing the mean-square error between the predicted Q-values and the target values derived from the Bellman equation:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} [(y - Q(s, a; \theta))^2] \quad (2)$$

where $y = r + \gamma \max_{a'} Q(s', a', \theta^-)$ is the target Q-value, θ^- represents the parameter of the target mesh, D is the experience playback buffer, and $U(D)$ denotes the homogeneous sampling of the experience from D .

3.2 GCN-driven feature extraction for fiber optic networks

The feature extraction part of this paper is divided into two parts: node feature extraction and branch feature extraction. In order to deal with the fiber network graph data effectively, the model adopt two layers to complete the extraction, aggregation and transformation of node features. GCN, as a classical type of deep neural network, successfully promotes the convolution operation from regular data (such as images) to graph-structured data, so that it can effectively deal with the node features in complex networks,, and its core formula is:

$$H^{(l+1)} = \sigma(D^{-1/2} \hat{A} D^{-1/2} H^{(l)} W^{(l)}) \quad (3)$$

where: $H^{(l+1)}$ is the output information of the convolutional processing of the l th layer GCN; $H^{(l)}$ is the node input information of the l th layer GCN, where $H^{(0)} = X$, $X = [X_1, X_2, \dots, X_n]^T$; $\hat{A} = A + I_n$ where A is the adjacency matrix of the graph and I_n is the unit matrix; D is the degree matrix; $W^{(l)}$ is the weight parameter matrix of the l th GCN layer; and $\sigma(\cdot)$ is the activation function. $L = D^{-\frac{1}{2}} \hat{A} D^{-\frac{1}{2}}$ is also known as the Laplace matrix, which serves to prevent numerical instability in the operation. Through the above two-layer GCN convolution operation, at time step t , the node feature representation h_t is finally obtained, and this feature vector can reflect the state characteristics of the node in the network at moment t .

The branch feature extraction network consists of fully connected layers, and its process depends on the output results of node feature extraction. . Remembering that at time step t , for branch l , the feature vector of node feature network output node b is b_t , then when the nodes at the two ends of the branch are node b_1 and node b_2 , the branch feature of branch l is $Y_{l,t} = (b_{l,t}, q_{l,t}, a_l, B_{b_1, b_2})$, the expression of the fully-connected layer is:

$$d_{l,t}^k = \begin{cases} \text{CONCAT}(h_{b_1,t}, h_{b_2,t}, Y_{l,t}) & \text{if } k = 0 \\ \sigma(W_s \cdot d_{l,t}^{k-1} + b_s) & \text{if } k > 0 \end{cases} \quad (4)$$

where $d_{l,t}^k$ is the branch l feature vector output from the k -th fully connected layer in the network at time t , and $k = 0$ is the splicing of the node features and branch features input to the network; W_s and b_s are the parameters of the fully connected layer.

After processing through the above fully connected layer, the branch feature vector $d_{l,t}$ is obtained and input to the output network. The output network successively link the single fully connected layer and the activation function, whoes main role is to calculate the risk value of branch l based on the branch feature vector. Remember that at time t , the branch feature of branch l is:

$$R_{l,t} = \text{ReLU}(\omega_s d_{l,t} + b'_s) \quad (5)$$

Where ω, b' is the single fully connected layer parameter and $\text{ReLU}(\)$ is the ReLU activation function.

3.3 GCN and DRL-based agent design for fiber optic networks

This section use advanced training of graph neural network policy models based on reinforcement learning. First, the Markov decision process of reinforcement learning is modeled, including modules such as state, action, and reward functions. Then, to improve the overall training efficiency, the initial model needs to be pre-trained by supervised learning methods. Finally, the policy model is trained using the DQN reinforcement learning algorithm to equip it to handle complex network environments.

In order to describe the above MDP decision model formulaically, the state space, and action space definitions are given below

3.3.1 State space

The state space includes the topology graph of the fiber optic communication network, link occupancy, node load and path state. As well as the link mediator, which indicates the number of paths that may cross a link. Specifically, we compute the number of link mesos in the following way: for each pair of nodes in the topology graph, we compute k candidate paths and maintain a per-link counter, which indicates the number of paths that cross the link. Thus, the number of meshes on each link is the number of end-to-end paths that cross the link divided by the total number of paths. To summarize, the state space consists of the network topology, link occupancy, node load, and path state together, where the number of link mediators reflects the number of paths that may cross the link.

Noting that the state of the model at time step t is S_t , then:

$$S_t = \{G_t, O_{l,t}, L_{n,t}, P_{p,t}, H_t, R_t\} \quad (6)$$

Where, at time step t , G_t denotes the connection state of the network topology graph; $O_{l,t}$ is the occupied bandwidth ratio of link l at time step t ; $L_{n,t}$ is the data traffic handled by node n ; and $P_{p,t}$ is the current availability and bandwidth reservation of path p , H_t is the node feature vector and R_t is the risk value of each branch.

3.3.2 Action space

In large-scale real networks, DRL agents often face the problem of a high-dimensional action space, and here we limit the number of candidate paths, significantly reducing the

otherwise potentially large action space by restricting the set of actions for each source-target node pair to k shortest paths. Doing so maintains a degree of routing flexibility and significantly reduces the number of actions that the model needs to explore and evaluate, thus improving learning efficiency and feasibility.

Considering the above factors, we restrict the set of actions to k candidate paths for each source-destination node pair. In order to maintain a good trade-off between the flexibility of routing traffic and the cost of evaluating all possible actions, we choose the shortest set of paths in each pair of source-target nodes ($k = 4$).

Furthermore, in this paper, we consider an additional link layer feature, namely the bandwidth allocated to a link after applying a routing action. This value corresponds to the current bandwidth requirement of the traffic request to be allocated. Similarly, links that are not included in the path selected by the action will have this feature set of θ . Since our environment has a finite number of traffic requests with a variety of discrete bandwidth requirements, we represent the allocated bandwidth using a one-hot thermally encoded vector of N elements, where N is the vector length.

For each pair of source-target nodes, the action set A_{sd} consists of k shortest paths, (these paths are selected as the best possible routing options based on a weighting operation of the number of shortest hops as well as the risk) i.e. $A_{sd} = \{P_{sd}^1, P_{sd}^2, P_{sd}^3, P_{sd}^k\}$

In summary, the action space A_a is designed in the model of this paper:

$$A_a = \bigcup_{s \in N_s, d \in N_d} \{P_{sd}^1, P_{sd}^2, P_{sd}^3, \dots, P_{sd}^k\} \quad (7)$$

Where N_s, N_d represent the set of source and target nodes in the network, respectively, and k is the number of predefined shortest paths.

3.3.3 Reward function

The reward function is designed as a reward component and a penalty component. The reward function is designed as follows:

$$\begin{aligned} r_t = & \omega_1 \times \text{BandwidthUtilization}(t) \\ & - \omega_2 \times \text{CongestionLevel}(t) \\ & + \omega_3 \times \text{PathSuccessRate}(t) \\ & + \omega_4 \times \text{RiskScore}_t \\ & - \omega_5 \times \text{HopCountFactor}_t \end{aligned} \quad (8)$$

where: $\text{BandwidthUtilization}(t)$ is the extent to which the network's bandwidth resources are utilized at time step t ; $\text{CongestionLevel}(t)$ is the degree of congestion in the network at time step t ; $\text{PathSuccessRate}(t)$ is the proportion of successful data transmission or routing requests in the network at time step t ; RiskScore is the risk assessment score, which is obtained by weighted summation of risk factors, with the formula $\text{RiskScore} = \omega_q \times q_{l,t} + \omega_p \times p_u + \omega_a \times (1 - a)$, where $\omega_q, \omega_p, \omega_a$ are the weights of the corresponding risk metrics. HopCountFactor_t is the Hop Count Factor, which is a hop-based attenuation function, with the formula $\text{HopCountFactor} = \omega_h \times e^{-\alpha \cdot \text{HopCount}}$ (ω_h is the weight of hop count). In order to improve the objectivity and reproducibility of the reward function, this paper uses an adaptive weighting mechanism based on information entropy to determine the weights of each indicator. Specifically, for the five indicators in the reward function, we quantify

their information content by calculating their discrete entropy in the training data, and define their weights accordingly:

$$\omega_i = \frac{1 - H_i}{\sum_j^5 (1 - H_j)} \quad (9)$$

$$H_i = - \sum_k p_{ik} \log(p_{ik}) \quad (10)$$

This represents the normalized probability distribution of the indicator. The mechanism ensures that indicators with high distinguishability play a dominant role in overall decision-making based on the principle that the higher the information content (lower entropy), the greater the weight, thereby constructing a more reasonable reward function structure.

3.3.4 DRL agent with PER and n -step rewards

In building the DRL agent, we first construct two core networks: an eval network, which is responsible for real-time decision-making and guiding the agent's actions in the environment, and a target network, which is designed to stabilize the learning process by providing a stable estimate of future rewards for value assessment. The main role of the eval net is to evaluate the Q value of each action in the current state and select the action with the largest Q value to guide the agent's decision. The role of the target net is to provide a relatively stable target Q value, which is used to compute the loss and update the parameters of the eval net, thus avoiding instability in the training process. Subsequently, we create an empirical replay buffer D , carefully designing each storage entry to contain exhaustive information: the current state s_t , the action taken a_t , the reward accumulated over the next n steps $R_{t:t+n}$, the new state s_{t+n} , and the initialization weights ω_t , in order to adequately capture the complex dynamics of sequential decision making.

Next, we meticulously configure a series of key parameters, including a learning rate α to control the pace of parameter updates, a discount factor γ to reflect the present value of future rewards, and a Prioritized Experience Replay (PER) mechanism that is introduced to optimize the learning efficiency and stability during the construction of the DRL agent. The PER mechanism allows the algorithm to non-uniformly sample experiences based on their importance, thus improving the learning efficiency. When implementing the PER mechanism, it is first necessary to prioritize each newly accumulated experience, which is determined based on the absolute Temporal Difference Error (TD) of the experience, i.e.

$$p_i = |TD_{error}| + \iota \quad (11)$$

Here, a small positive number ι is used to ensure that even experiences with zero TD error have some non-zero priority, thus ensuring that all experiences have a chance to be learned by replay. Once the priority of each experience is determined, the probability of each experience being sampled can be calculated based on these priorities. The sampling probabilities are assigned proportionally, i.e.,

$$P(i) = \frac{p_i^{\alpha'}}{\sum_j p_j^{\alpha'}} \quad (12)$$

Here α' is a hyperparameter that controls the degree of skewness of the sampling probability. At that time, PER degenerates into Uniform Experience Replay (UER). As it increases, the PER mechanism will be more inclined to select those experiences with larger TD errors.

Due to the uneven sampling probability caused by the PER mechanism, a weight ω_i needs to be computed to compensate for the sampling bias and to ensure unbiased learning objectives:

$$\omega_i = \frac{(\frac{1}{N \cdot P(i)})^\beta}{\max_j (\frac{1}{N \cdot P(j)})^\beta} \quad (13)$$

where N is the total number of experiences in the experience pool and β is another hyperparameter to control the scale of the weights. As training proceeds, β is gradually increased to 1 to gradually reduce the bias effect of the weights. After each update of the network, the TD error of each experience needs to be recalculated and its priority in the experience pool updated accordingly. This ensures that the algorithm always focuses on those experiences that are most valuable.

In the state feature extraction phase, we adopt an enhanced GCN model, which refines the deep feature vector h^{k+1} of each node on the basis of the graph data G_t and edge features e_{uv} at a given time step t , and then integrates them to form a comprehensive state representation s'_t .

Unlike the traditional DQN algorithm, the action selection and execution mechanism of this algorithm is further upgraded, which is no longer simply based on immediate feedback, but instead waits for n steps and then cumulatively calculates the sum of rewards, R_t^n , from t to $t+n$, in combination with the value prediction of the future state, which is provided by the target network, as a measure of the long term gain, which is formulated as:

$$R_t^n = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n Q(s_{t+n}, \arg \max_a Q(s_{t+n}, a; \Theta^-); \theta^-) \quad (14)$$

In the subsequent priority assignment step, for each newly accumulated experience, we determine the priority p_i based on the absolute value of its cumulative reward plus a small constant ι . This ensures that all experiences have a chance to be reached by the learning algorithm, and adjusts the position of the experience in the pool accordingly to achieve effective experience replay.

During the experience playback process, we implement priority sampling, which assigns different sampling probabilities $P(i)$ based on the importance of the experience to ensure that high-value experiences are frequently learned, and calculates the weight ω_i for each sample to balance the sampling bias. Specifically, the weight ω_i is calculated by the importance sampling formula, reflecting its relative rarity in the overall experience distribution. Ultimately, the network update session adopts the experience of the sampled batches through an adjusted loss function:

$$L = \sum_i \omega_i \left(R_i^n + \gamma \max_{a'} Q(s'_{i+n}, a'; \theta^-) - Q(s_i, a_i; \theta) \right)^2 \quad (15)$$

to drive the optimization of network parameters θ . And, based on the newly estimated Q-value, we update the empirical priority again, usually based on the absolute value of the TD error, in order to continuously iteratively optimize the learning strategy, and ultimately achieve a significant improvement in agent performance.

The pseudo-code of the training process is given below, showing in detail the steps of each training round and their logical relationships.

```

1: for episode = 1 to  $E$  do
2:   Reset environment and obtain initial state  $S_0$ .
3:   for  $t = 0$  to  $T$  do
4:     a. Node feature extraction: Extract node feature  $h_t$  from  $G_t$  using GCN.
5:     b. Branch feature extraction: Compute branch feature vectors  $d_{l,t}$  using
       $h_t$  and branch attributes  $Y_{l,t}$ .
6:     c. Action space definition: Define action space  $A_a$  based on  $k$  shortest
      paths for each source-destination pair.
7:     d. Action selection: Select action  $a_t$  using  $\epsilon$ -greedy policy from  $Q(s, a; \theta)$ .
8:     e. Execute action: Execute  $a_t$ , observe reward  $r_t$  and next state  $S_{t+1}$ .
9:     f. Store experience: Store transition  $(S_t, a_t, r_t, S_{t+1})$  in replay buffer  $D$ .
10:    if  $|D| > \text{batch size } B$  then
11:      i. Sample mini-batch: Sample  $\{(S_j, a_j, r_j, S_{j+1})\}$  from  $D$  using PER
      probabilities  $P(i)$ .
12:      ii. Compute target Q-values: For each sample, compute the  $n =$ 
      5-step return  $R_j^n$  (see Eq. (2) for details).
13:      iii. Compute TD error: Compute  $\delta_j = |R_j^n - Q(S_j, a_j; \theta)|$ .
14:      iv. Update priorities: Update priority  $p_i$  in  $D$  based on  $\delta_j$  (see Eq.
      (3) for details).
15:      v. Compute importance sampling weights: Compute  $\omega_i$  for each
      sample in mini-batch (see Eq. (4) for details).
16:      vi. Update evaluation network parameters: Minimize the loss  $L$ 
      (see Eq. (5) for details).
17:      vii. Update target network parameters: Synchronize  $\theta^- \leftarrow \theta$  every
       $C = 5$  steps to stabilize training.
18:    end if
19:  end for
20: end for

```

Algorithm 1 Main Loop of DRL Training

4 Analysis of experimental results/simulation examples

To verify the effectiveness of the model in this paper, we use the data of power communication network in a province of State Grid for training and testing. We take part of this dataset nodes include a total of 1200 physical nodes involved in different services in the field of electric power communication, each node includes node type, equipment status, capacity, packet loss rate, and the level of emergency plan supported by this communication node as well as the bandwidth of the fiber optic link, the quality of the signal, and the configuration of whether there is an alternate path as a binary marking of the emergency measures.

4.1 Algorithm parameterization

The experimental parameter settings in this paper are shown in Table 2.

Table 2 Hyperparameter Settings

Parameter	Setting
Learning rate α	0.001
Discount factor γ	0.92
Exploration rate ϵ	0.1
Replay buffer size D	10,000
Update frequency C	5
Target network update rate τ	0.002
PER decay rate α'	0.6
PER weight influence factor β	0.4
Priority minimum value ι	0.0001
Number of available shortest paths k	4
Batch size B	128
multi-step return length n	5

4.2 Algorithm scheduling results

4.2.1 Performance evaluation

In this evaluation experiment, we compare our RQGRL agent with state-of-the-art DRL-based solutions. We consider three traffic demand types (STM-1, STM-4, and STM-16)) [26]. When a DRL agent correctly assigns a demand, it is immediately rewarded with the bandwidth of the current traffic demand if the demand is correctly assigned and zero otherwise. We define that a demand is considered to be successfully assigned if the links on all paths chosen by the DRL agent have enough available capacity to carry such a demand. Conversely, if the demand is not successfully allocated, the attempt is considered a failure and moves to the next time step or starts a new simulation cycle.

To validate the effectiveness of the proposed RQGRL method, this study selected state-of-the-art deep reinforcement learning models as comparison baselines. All baseline models were trained and tested in the same simulation environment, with the specific methods as follows:

GCN-DRL model [27]: A fault recovery decision-making method based on graph convolutional networks (GCN) and deep Q-learning (DQN). It uses two layers of GCN to extract node features (device status, capacity, etc.) and selects the shortest path via DQN.

RiskGNN model [28]: A dynamic risk aggregation graph neural network that evaluates network risk through hierarchical propagation of node features (packet loss rate, emergency response plan level).

Safe-DRL model [29]: A safety-constrained reinforcement learning method for integrated energy systems, which restricts the action space through a predefined safety guidance function.

TreeSearch-DRL model [30]: A combined unit optimization method integrating Monte Carlo Tree Search (MCTS) and DRL, which reduces the exploration complexity of high-dimensional action spaces through tree pruning strategies. Although it performs stably in long-term planning, the decision delay can reach up to 162 ms (due to the time-consuming search tree backtracking).

PPO-OPF model [31]: An optimal power flow solver based on Proximal Policy Optimization (PPO), which uses convex relaxation techniques to convert non-convex problems into second-order cone programming (SOCP) to improve computational efficiency.

As shown in Table 3, the bandwidth allocation efficiency of RQGRL reached 93.2%, an improvement of 3.7 percentage points over the next-best RiskGNN method (89.5%)

Table 3 Performance evaluation results

Method	Bandwidth efficiency (%)	Fault recovery time (ms)	Risk avoidance rate (%)	Cross-topology error (%)
RQGRL	93.2	85	92.5	8.7
GCN-DRL	87.6	112	83.4	15.2
RiskGNN	89.5	98	88.9	12.3
Safe-DRL	84.3	150	85.7	18.6
TreeSearch	88.9	162	79.8	22.1
PPO-OPF	84.1	–	72.3	–

and nearly 10% over the traditional convex relaxation method PPO-OPF (84.1%). This improvement stems from RQGRL's unique edge weighting mechanism—by dynamically integrating link parameters, signal quality, and contingency plan level features through a graph convolutional network, the comprehensive risk score is encoded as edge weights (Equation (5)), thereby simultaneously optimizing resource utilization and risk avoidance in path planning. In contrast, GCN-DRL relies solely on node state aggregation, ignoring the dynamic characteristics of the link layer; while PPO-OPF, due to its convex approximation of non-convex problems, results in a constrained solution space, making it difficult to capture the multi-objective trade-off relationships in real networks.

In terms of fault recovery capability, RQGRL completes path switching in just 85 ms, a 43.3% improvement over the safety-constraint-driven Safe-DRL method (150 ms). This is attributed to its Priority Experience Replay (PER) mechanism (Equation (13)), which enhances the agent's learning efficiency for alternative paths of critical vulnerable links by weighted sampling historical high-risk link fault cases. In contrast, Safe-DRL relies on static capacity threshold constraints on the action space, unable to dynamically perceive the propagation risks of sudden failures, leading to delayed responses. Additionally, RQGRL achieves a risk avoidance rate of 92.5%, outperforming RiskGNN (88.9%) and the tree search pruning method TreeSearch (79.8%) by 3.6% and 12.7%, respectively. This gap highlights the superiority of edge weight quantization over node-centric risk modeling: RQGRL precisely identifies and avoids vulnerable links with high packet loss rates and low backup paths by dynamically updating link risk weights (Equation (5)), while RiskGNN's node aggregation strategy struggles to capture cross-link risk correlations, and TreeSearch lacks explicit risk modeling capabilities altogether.

In dynamic topology scenarios, RQGRL's cross-topology generalization error is only 8.7%, a reduction of 42.8% and 29.3% compared to GCN-DRL (15.2%) and RiskGNN (12.3%), respectively. This indicates that the edge-weighted hierarchical feature propagation mechanism (Equation (3)) enables the model to adapt to the random addition or removal of nodes and links while maintaining the semantic consistency of the topological structure. For example, when 20% of the nodes are randomly removed, RQGRL can still maintain a bandwidth allocation efficiency of over 90% by dynamically adjusting the degree and risk weights of the remaining links, while GCN-DRL suffers a performance loss of up to 15.2% due to its over-reliance on fixed node features. Notably, PPO-OPF and TreeSearch completely fail in this scenario (marked as “–”), with the former constrained by the static assumptions of the convex relaxation model and the latter unable to meet real-time decision-making requirements due to the high latency of tree search. These results collectively validate RQGRL's technical breakthrough in complex dynamic

environments—achieving synergistic optimization of efficiency, safety, and generalization through non-convex dynamic programming and graph network-driven adaptive risk quantification (Fig. 2).

4.2.2 Link failure recovery capability

This section is used to evaluate whether this method can effectively identify and avoid risk factors in the network. The risk impact factor chosen in this paper is $F = \{\text{link congestion, link failure}\}$. Link congestion and link failure can exist together or separately in the same scenario, depending on the operational state of the system and external perturbations. The indicator sets f_1, f_2, f_3 are subsets of F . f_1 means that the intelligent body only considers the link congestion indicator. f_2 means that the intelligent body only considers the link failure indicator. $f_3 = \{f_{31}, f_{32}\}$ means that the intelligent body considers both indicators at the same time, where f_{31} is the link congestion indicator and f_{32} is the link failure indicator in the case of f_3 .

In this evaluation, we consider a range of scenarios that may experience up to 10 link failures. Previous work showed that real-world network topologies change during (e.g., due to link failures) [32, 33]. These changes in network connectivity are unpredictable, and they have a significant impact on protocol convergence [32] or achieving network optimization goals [1]. Therefore, the task of the RQGRL agent is to find new routing configurations to avoid the affected links while still maximizing the total allocated bandwidth. We perform the following operations in the topology of the communication network from the State Grid Power: (1) Gradually increase the traffic through these links for a specific period of time until it reaches or exceeds 90% of the link capacity, simulating a highly congested state. (2) Randomly select links in the network for failure simulation, which can be both transient failures (recovery of a link after a brief interruption) and permanent failures (complete failure of a link), with each failure affecting one to many links. We compare the scores (i.e., allocated bandwidth) achieved by the RQGRL agents with respect to the theoretical flow model. Figure 3 shows the relationship between the average score (y-axis) and the number of link failures (x-axis). We observe that RQGRL consistently achieves higher average scores than the other five state-of-the-art algorithms, exhibiting a gradual decline as the number of failures increases. Even when the network experiences 10 concurrent link failures, RQGRL maintains a bandwidth allocation capability of 452 points—an 11.3% improvement over the next-best Safe-DRL method (406 points)—validating its robust performance in high-risk environments. Specifically, in low-fault scenarios (0-4 failures), RQGRL precisely detects local congestion

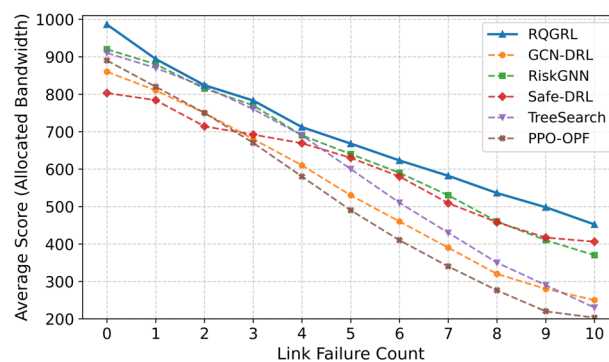


Fig. 2 Evaluate a use case with link failures

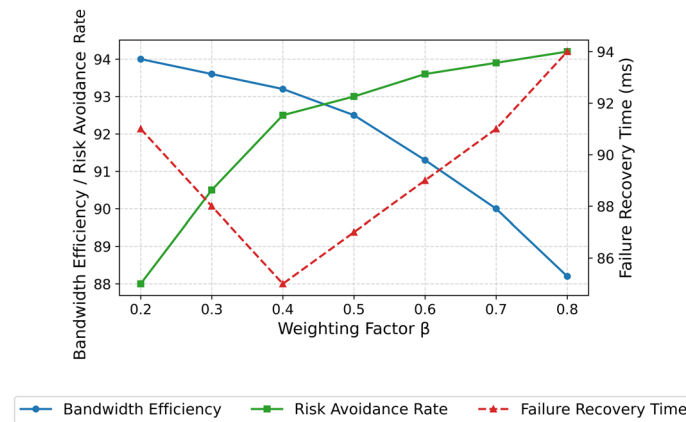


Fig. 3 Evaluate a use case with link failures

risks through its edge weight quantization mechanism. Its scores (e.g., 783 points at 3 failures) significantly outperform RiskGNN (770 points) in low-fault scenarios (0–4 failures). In high-fault scenarios (5–10 failures), its dynamic path re-planning capability effectively circumvents the performance collapse of the TreeSearch method (230 points at 10 failures) caused by action space explosion. Translated with DeepL.com (free version).

Experimental results demonstrate that RQGRL's dynamic risk quantification mechanism and graph network generalization capability form the core of its performance advantage: the real-time edge weight update strategy (Equation 5) enables agents to synchronously respond to link congestion and failures, while priority-based experience replay (Equation 13) ensures long-term benefit optimization by reinforcing learning in high-risk scenarios. These innovations endow RQGRL with significant application potential in dynamic complex systems such as State Grid's power communication networks. Translated with DeepL.com (free version).

4.2.3 Hyperparameter sensitivity experiments

To further analyze the impact of the weighting factor β on model performance, this section designs a hyperparameter sensitivity experiment. As the weighting parameter for the risk-averse component of the reward function, the value of β directly influences the agent's strategy for balancing resource optimization and risk avoidance. If the value is low, the agent tends to prioritize bandwidth efficiency, potentially neglecting potential risks in the link; conversely, when the value is high, the agent is more inclined to avoid high-risk paths, but this may lead to reduced resource utilization. Therefore, setting an appropriate value is crucial for improving overall performance. In the experiment, while keeping other hyperparameters constant, β was set to 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, and the RQGRL model was trained and tested accordingly. The three key performance metrics—bandwidth allocation efficiency, fault recovery time, and risk avoidance rate—were compared and evaluated, with the results shown in Figure 3. It can be observed that as β increases from 0.2 to 0.4, the risk avoidance rate continues to improve, while bandwidth efficiency remains at a high level. At this point, the model achieves optimal overall performance: bandwidth allocation efficiency of 93.2%, fault recovery time of 85 ms, and risk avoidance rate of 92.5%. When β is further increased to 0.7 and 0.8, although the risk avoidance rate continues to rise, the bandwidth efficiency decreases significantly,

and the fault recovery time is prolonged, indicating that the agent strategy becomes overly conservative at high values, resulting in reduced resource utilization efficiency. Therefore, considering the three factors of resource utilization, fault response, and risk control, the optimal setting is ultimately selected in this paper.

5 Conclusion

In this paper, we design a set of novel self-optimizing control strategies for the optimization and control challenges of electric power communication networks, focusing on breaking through the limitations of traditional methods that neglect risk assessment. The core of the strategy lies in combining deep reinforcement learning with graph neural networks, dynamically adjusting the network strategy through the risk quantization method, and realizing the efficient allocation of communication resources and intelligent decision-making for path planning. The experimental part verifies the effectiveness of the model, and by comparing with the current state-of-the-art DRL system, theoretical fluid model and load-balanced routing strategy, it is shown that the proposed RQGRL model achieves a significant improvement in bandwidth allocation efficiency, and the model maintains high bandwidth utilization and network stability when dealing with link failures and high congestion, which proves its superiority in complex network environments. However, this algorithm still has significant limitations: it relies heavily on data integrity, and when critical network state data is missing, it can easily lead to risk quantification bias and reduced accuracy in path planning decisions; Scalability is inadequate. When confronted with ultra-large-scale network topologies, the model's feature extraction and empirical sampling efficiency significantly decrease, making it difficult to meet real-time decision-making demands. Additionally, the action space design is relatively rigid, failing to fully account for dynamic adjustments and multi-path resource allocation possibilities. Under extreme traffic scenarios, there remains room for optimization in resource utilization efficiency. Future research will focus on enhancing data robustness, adapting to large-scale networks, and optimizing the action space to further improve the model's applicability in practical engineering scenarios.

Author contributions

Conceptualization: Y.J., C.S.; Methodology: Y.J., Y.W.; Formal analysis and investigation: A.T.; Writing—original draft preparation: A.T., Y.Z.; Writing—review and editing: Y. W., C.S.; Funding acquisition: Y.Z.; Resources: C.S.; Supervision: Y.Z.

Funding

This work was supported by the Science and Technology Project of State Grid Shandong Electric Power Company: "Research on Self-optimizing Control Technology based on Quantitative Assessment of Electric Power Communication Risk" (Grand No. 520627240007).

Data availability

No datasets were generated or analysed during the current study.

Code availability

Not applicable.

Declarations

Ethics approval and consent to participate

There are no potential Conflict of interest and no animal or human research is involved in this paper.

Consent for publication

The participant has consented to the submission of this manuscript to the journal.

Competing interests

The authors have no relevant financial or non-financial interests to disclose.

Published online: 14 November 2025

References

1. Hartert R, Vissicchio S, Schaus P, Bonaventure O, Filis C, Telkamp T, et al. A declarative and expressive approach to control forwarding paths in carrier-grade networks. *SIGCOMM Comput Commun Rev.* 2015;45:15–28. <https://doi.org/10.1145/2829988.2787495>.
2. Knight S, Nguyen HX, Falkner N, Bowden R, Roughan M. The internet topology zoo. *IEEE J Select Areas Commun.* 2011;29:1765–75. <https://doi.org/10.1109/JSAC.2011.1111002>.
3. Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, et al. Human-level control through deep reinforcement learning. *Nature.* 2015;518:529–33. <https://doi.org/10.1038/nature14236>.
4. Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A, Lanctot M, Sifre L, Kumaran D, Graepel T. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. (2017). <https://arxiv.org/abs/1712.01815>
5. Feamster N, Rexford J Why (and how) networks should run themselves. In: *Proceedings of the 2018 Applied Networking Research Workshop*. Association for Computing Machinery, New York, NY, USA, p 20 (2018). <https://doi.org/10.1145/3232755.3234555>
6. Wang M, Cui Y, Wang X, Xiao S, Jiang J. Machine learning for networking: workflow, advances and opportunities. *IEEE Netw.* 2018;32:92–9. <https://doi.org/10.1109/MNET.2017.1700200>.
7. Mestres A, Rodríguez-Natal A, Carner J, Barlet-Ros P, Alarcón E, Solé M, Muntés-Mulero V, Meyer D, Barkai S, Hibbett MJ. Knowledge-defined networking. *arXiv*<https://arxiv.org/abs/1606.06222> (2016)
8. Kalmbach P, Zerwas J, Babarczy P, Blenk A, Kellerer W, Schmid S. Empowering self-driving networks. In: *Proceedings of the Afternoon Workshop on Self-Driving Networks. SelfDN 2018*. Association for Computing Machinery, New York, NY, USA 2018:8–14. <https://doi.org/10.1145/3229584.3229587>
9. Feng B, Hu Y, Huang G, et al. Review on optimization methods for new power system dispatch based on deep reinforcement learning. *Autom Electr Power Syst.* 2023;47:187. <https://doi.org/10.7500/AEPS20220228002>.
10. Chen X, Guo J, Zhu Z, Proietti R, Castro A, Yoo SJB. Deep-rmsa: A deep-reinforcement-learning routing, modulation and spectrum assignment agent for elastic optical networks, 2018:4–2 <https://doi.org/10.1364/OFC.2018.W4F.2>
11. Almasan P, Suárez-Varela J, Rusek K, Barlet-Ros P, Cabellos-Aparicio A. Deep reinforcement learning meets graph neural networks: exploring a routing optimization use case. *Comput Commun.* 2022;196:184–94. <https://doi.org/10.1016/j.comcom.2022.09.029>.
12. Wang X, Zhang L, Ren X, et al. Optimal scheduling of integrated energy systems by fusing a graph neural network model and reinforcement learning. *Power Syst Protect Control.* 2023;51(24):102–10.
13. Ensley MR. Toward a theory of situation awareness in dynamic systems. *Human Fact.* 1995;37:85–104.
14. Zhigang Z, Chongqing K. Challenges and prospects of building a new type of power system under carbon neutralization targets. *Proc CSEE.* 2022;42:2806–19. <https://doi.org/10.13334/j.0258-8013.pcsee.220467>.
15. Sheng, Y., Cai, D., Wang, E., Chen, J.: Transient stability assessment of power systems based on semi-supervised active learning, 2023; pp 758–763. <https://doi.org/10.1109/AEES56888.2023.10114094>
16. Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G. The graph neural network model. *IEEE Trans Neural Netw.* 2009;20:61–80. <https://doi.org/10.1109/TNN.2008.2005605>.
17. Li Y, Tarlow D, Brockschmidt M, Zemel R. Gated graph sequence neural networks (2017)
18. Veličković P, Cucurull G, Casanova A, Romero A, Lió P, Bengio Y. Graph attention networks (2018)
19. Rusek K, Suárez-Varela J, Almasan P, Barlet-Ros P, Cabellos-Aparicio A. Routenet: Leveraging graph neural networks for network modeling and optimization in SDN. *CoRR* [abs/1910.01508](https://arxiv.org/abs/1910.01508) (2019) [arXiv https://arxiv.org/abs/1910.01508](https://arxiv.org/abs/1910.01508)
20. Xu Z, Tang J, Meng J, Zhang W, Wang Y, Liu CH, Yang D. Experience-driven networking: A deep reinforcement learning based approach, 2018; pp1871–1879. <https://doi.org/10.1109/INFOCOM.2018.8485853>
21. Zhang P, Chen Y, Wang G, et al. Fault recovery decision of distribution network based on graph reinforcement learning. *Autom Electric Power Syst.* 2024;48(2):151–8.
22. Guo C, Liu Z, Liu Y, et al. GNN and RL based power system risk situation perception. *Power Syst Clean Energy.* 2023;39(12):41–9.
23. Sutton RS, Barto AG. Reinforcement learning: an introduction. Cambridge, MA, USA: MIT Press; 2018.
24. Suárez-Varela J, Mestres A, Yu J, Kuang L, Feng H, Cabellos-Aparicio A, et al. Routing in optical transport networks with deep reinforcement learning. *J Opt Commun Netw.* 2019;11:547–58. <https://doi.org/10.1364/JOCN.11.000547>.
25. Watkins CJCH, Dayan P. Q-learning. *Mach Learn.* 1992;8:279–92. <https://doi.org/10.1007/BF00992698>.
26. ITU-T: Recommendation g.707 : Network node interface for the synchronous digital hierarchy (sdh). In: *Recommendation, International Telecommunication Union* 2019. <https://www.itu.int/rec/T-REC-G.707>
27. Feng B, Hu YJ, Huang G, et al. Review of novel power system dispatching optimization methods based on deep reinforcement learning. *Autom Electric Power Syst.* 2023;47(17):187–99. <https://doi.org/10.1109/TPWRS.2023.17019>.
28. Zhang ZG, Kang CQ. Challenges and prospects for constructing the new-type power system towards a carbon neutrality future. *Proc CSEE.* 2022;42(8):2806–18. <https://doi.org/10.13334/j.0258-8013.pcsee.220467>.
29. Qiu D, Dong Z, Zhang X, et al. Safe reinforcement learning for real-time automatic control in a smart energy-hub. *Appl Energy.* 2022;309:118403. <https://doi.org/10.1016/j.apenergy.2021.118403>.
30. Mars PD, O'Sullivan A. Applying reinforcement learning and tree search to the unit commitment problem. *Appl Energy.* 2021;302:117519. <https://doi.org/10.1016/j.apenergy.2021.117519>.
31. Zhou YH, Lee WJ, Diao R, et al. Deep reinforcement learning based real-time ac optimal power flow considering uncertainties. *J Modern Power Syst Clean Energy.* 2022;10(5):1098–109. <https://doi.org/10.35833/MPCE.2020.000885>.
32. Francois P, Filis C, Evans J, Bonaventure O. Achieving sub-second IGP convergence in large IP networks. *SIGCOMM Comput Commun Rev.* 2005;35:35–44. <https://doi.org/10.1145/1070873.1070877>.
33. Jain S, Kumar A, Mandal S, Ong J, Poutievski L, Singh A, et al. B4: experience with a globally-deployed software defined wan. *SIGCOMM Comput Commun Rev.* 2013;43:3–14. <https://doi.org/10.1145/2534169.2486019>.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.